

Three tokens, small integers, one attention head

On paper, with a calculator for the exponentials. Compute how much each word attends to the others, then change one key and predict what moves before you recompute.

Paper exercise

Calculator allowed

Integers throughout

Check cells afterwards

Before you start

The sentence

1 · the row for `clinic`

2 · the row for `needs`

3 · read the whole table

4 · change one key

Reflection

Before you start

Work on the printed sheet. The notebook holds check cells for after you have written your numbers — not before.



By the end of this session you can

- Compute scaled dot-product scores between queries and keys.
- Turn a row of scores into attention weights with a softmax.
- Form the output as a weighted sum of value vectors.
- Predict which weights move when one key changes.

SESSION 25

The sentence

clinic → **needs** → **medicine**, three tokens, each already turned into a query, a key and a value vector with two components:

token	query q	key k	value v
clinic	[1, 0]	[1, 0]	[2, 0]
needs	[1, 1]	[1, 1]	[1, 1]
medicine	[0, 1]	[0, 2]	[0, 3]

The vectors have dimension $d = 2$, so every score is divided by $\sqrt{2} \approx 1.414$.

QUESTION 1 · 8 MINUTES

the row for `clinic`

1

1. Dot product of clinic's query with **each** of the three keys: three integers.
2. Divide each by $\sqrt{2}$.
3. Softmax: take e to the power of each scaled score, then divide each by their sum. The three weights must add to 1.
4. Output = $\text{weight}_1 \cdot v(\text{clinic}) + \text{weight}_2 \cdot v(\text{needs}) + \text{weight}_3 \cdot v(\text{medicine})$. Two numbers.

QUESTION 2 · 7 MINUTES

the row for `needs`

2

The same four steps with the query [1, 1]. Before computing: which key do you expect needs to attend to most, and why?

check cell

PYTHON

```
# check cell - Questions 1 and 2
import numpy as np
Q = np.array([[1., 0.], [1., 1.], [0., 1.]])
K = np.array([[1., 0.], [1., 1.], [0., 2.]])
V = np.array([[2., 0.], [1., 1.], [0., 3.]])

scores = Q @ K.T / np.sqrt(2)
weights = np.exp(scores) / np.exp(scores).sum(axis=1, keepdims=True)
output = weights @ V
for name, s, w, o in zip(["clinic", "needs", "medicine"], scores, weights, output):
    print(f"{name:<9} scores {np.round(s, 3)} weights {np.round(w, 3)} output {np.round(o, 3)}")
```

what it should print

OUTPUT

```
clinic    scores [0.707 0.707 0.   ] weights [0.401 0.401 0.198] output [1.203 0.994]
needs     scores [0.707 1.414 1.414] weights [0.198 0.401 0.401] output [0.797 1.604]
medicine  scores [0.   0.707 1.414] weights [0.14  0.284 0.576] output [0.564 2.012]
```

QUESTION 3 · 4 MINUTES

read the whole table

3

Run or read the check cell's third row. Why does `medicine` put more than half of its attention on itself? Point at the specific numbers in Q and K that cause it.

QUESTION 4 · 6 MINUTES

change one key

4

The key for `medicine` changes from `[0, 2]` to `[2, 0]`.

Before any arithmetic, predict: which row of the weights changes most, and in which direction does its largest weight move? Write the prediction down, then recompute that one row.



Do not say a token *decides* or *understands*. Name the quantity that changed: a dot product, and therefore a score, and therefore a weight.

check cell

PYTHON

```
# check cell - Question 4
K2 = K.copy(); K2[2] = [2., 0.]
s2 = Q @ K2.T / np.sqrt(2)
w2 = np.exp(s2) / np.exp(s2).sum(axis=1, keepdims=True)
print("weights before:\n", np.round(weights, 3))
print("weights after:\n", np.round(w2, 3))
```

what it should print

OUTPUT

```
weights before:
[[0.401 0.401 0.198]
 [0.198 0.401 0.401]
 [0.14  0.284 0.576]]
weights after:
[[0.248 0.248 0.503]
 [0.198 0.401 0.401]
 [0.248 0.503 0.248]]
```

Reflection



Write it in the notebook, in your own words

Write the chain **token** → **token ID** → **embedding** → **query, key, value** → **attention-weighted output** and, under each arrow, one sentence saying what happens there. Session 24 gave you the first two arrows.